HostingLint Rules Roadmap: Comprehensive Implementation Plan

Document Version: 1.0

Date: February 8, 2026

Purpose: Technical blueprint for expanding HostingLint from 28 to 50+ rules for NLnet NGI Zero Commons Fund grant application

---

Executive Summary

This document provides a comprehensive roadmap for expanding HostingLint's static analysis capabilities from the current **28 rules** to **50+ rules** as part of the NLnet grant Milestone 1 (Rule engine expansion).

Current Distribution:

- PHP/WHMCS: 13 rules
- Perl/cPanel: 8 rules
- OpenPanel: 5 rules
- Cross-platform: 2 rules

Proposed Expansion:

- **22+ new rules** targeting high-impact security vulnerabilities, modern compatibility issues (PHP 8.2/8.3, WHMCS 8.11-8.13, cPanel UAPI), and Docker container security
- Priority focus on vulnerabilities with documented CVEs from 2024-2025
- 100% test coverage requirement for all new rules

Table of Contents

1. [Current Rule Inventory](#1-current-rule-inventory-28-rules)
2. [Proposed New Rules](#2-proposed-new-rules-by-priority)
3. [Rule Implementation Specifications](#3-rule-implementation-specifications)
4. [Technical Implementation Strategy](#4-technical-implementation-strategy)
5. [Grant Milestone Mapping](#5-grant-milestone-mapping)
6. [Testing Strategy](#6-testing-strategy)
7. [Performance Considerations](#7-performance-considerations)
8. [Community Contribution Model](#8-community-contribution-model)
9. [Appendices](#9-appendices)

1. Current Rule Inventory (28 Rules)

1.1 PHP/WHMCS Rules (13 rules)

Compatibility (4 rules)

Rule ID	Description	Severity	Min PHP Version
`php-compat-each`	each() removed in PHP 8.0	error	8.0
`php-compat-create-function`	create_function() removed in PHP 8.0	error	8.0
`php-compat-mysql-functions`	mysql_* functions removed in PHP 7.0	error	7.0
`php-compat-curly-braces`	Curly brace array access deprecated PHP 7.4	error	7.4

Security (3 rules)

Rule ID	Description	Severity	Detection Pattern
`security-sql-injection`	SQL injection via unsanitized input	error	$_GET/$_POST/$_REQUEST in SQL
`security-xss`	XSS via unescaped echo/print	error	echo/print with user input
`security-path-traversal`	Path traversal in file operations	error	User input in file functions

WHMCS-Specific (6 rules)

Rule ID	Description	Severity	Category
`whmcs-metadata`	Missing MetaData() function (required WHMCS 8.0+)	warning	compatibility
`whmcs-deprecated`	Deprecated WHMCS functions (select_query, etc.)	warning	compatibility
`whmcs-hook-error-handling`	Missing try/catch in hook callbacks	warning	best-practice
`whmcs-config-function`	Missing _Config() in provisioning modules	warning	compatibility
`whmcs-license-check`	Missing license validation	info	best-practice
`whmcs-return-format`	Incorrect return format in module functions	warning	best-practice

1.2 Perl/cPanel Rules (8 rules)

Compatibility (2 rules)

Rule ID	Description	Severity
`perl-cpanel-api-version`	Deprecated cPanel API1 usage	warning
`perl-deprecated-modules`	Deprecated Perl modules (CGI, Switch, etc.)	warning

Security (3 rules)

Rule ID	Description	Severity
`perl-security-taint`	Untainted external input in system/exec	error
`perl-file-permissions`	Insecure file permissions (chmod 777)	warning
`perl-input-validation`	CGI params without validation	warning

Best Practices (2 rules)

Rule ID	Description	Severity
`perl-strict-warnings`	Missing use strict/warnings	warning
`perl-error-handling`	Critical operations without eval/die	warning

1.3 OpenPanel Rules (5 rules)

Compatibility (1 rule)

Rule ID	Description	Severity
`openpanel-api-versioning`	Missing apiVersion in manifest	warning

Security (2 rules)

Rule ID	Description	Severity
`openpanel-security-capabilities`	Excessive Docker capabilities/privileged mode	error
`openpanel-cli-validation`	Missing input validation in CLI scripts	warning

Docker Best Practices (2 rules)

Rule ID	Description	Severity
`openpanel-dockerfile`	Missing USER directive or HEALTHCHECK	warning
`openpanel-resource-limits`	Missing resource limits in docker-compose	warning

1.4 Cross-Platform Rules (2 rules)

Rule ID	Description	Severity	Platforms
`security-hardcoded-credentials`	Hardcoded passwords/API keys	error	all
`security-eval-usage`	Usage of eval() function	warning	all
`best-practice-todo-fixme`	TODO/FIXME comments	info	all

---

2. Proposed New Rules (By Priority)

Priority 1: High-Impact Security Rules (8 rules)

Based on 2024-2025 vulnerability trends and CVE analysis:

P1.1 PHP Security Rules (3 rules)

Rule ID	Description	Justification	CVE Reference
`security-php-deserialization`	Insecure unserialize() usage	Remote code execution vector, OWASP Top 10	CVE-2024-4577 context
`security-php-ssrf`	Server-side request forgery via user-controlled URLs	Common in WHMCS API integrations	-
`security-php-weak-crypto`	Weak crypto algorithms (MD5, SHA1 for passwords)	Password cracking, NIST deprecation	-

Implementation Details:

```
// security-php-deserialization
Pattern: \bunserialize\s*\(\s*\$(?:_GET|_POST|_REQUEST|_COOKIE)
Message: "Insecure deserialization: never unserialize() untrusted data"
Fix: "Use JSON or validated serialization formats"

// security-php-ssrf
Pattern: \b(?:file_get_contents|curl_exec|fopen)\s*\([^)]*\$(?:_GET|_POST|_REQUEST)\[
Message: "Potential SSRF: user input controls URL in HTTP request"
Fix: "Validate URLs against whitelist, use allow_url_fopen = Off"

// security-php-weak-crypto
Pattern: \b(?:md5|sha1)\s*\(\s*\$(?:password|passwd|pwd)
Message: "Weak password hashing: MD5/SHA1 are not suitable for passwords"
Fix: "Use password_hash() with PASSWORD_ARGON2ID or PASSWORD_BCRYPT"
```

P1.2 Perl Security Rules (2 rules)

Rule ID	Description	Justification
`security-perl-sql-injection`	SQL injection via string interpolation	Not covered by current taint rule

Rule ID	Description	Justification
`security-perl-cmd-injection`	Enhanced command injection detection	Expand beyond current taint patterns

```
# security-perl-sql-injection
Pattern: \$dbh->(?:do|selectall|selectrow).*\$
Message: "Potential SQL injection: use placeholders with execute()"
Fix: "my $sth = $dbh->prepare('... WHERE id = ?'); $sth->execute($id);"

# security-perl-cmd-injection
Pattern: qx\{.*\$|system\s*\([^)]*\.\s*\$
Message: "Command injection via string concatenation"
Fix: "Use IPC::Run or pass arguments as list: system('cmd', $arg1, $arg2)"
```

P1.3 OpenPanel Security Rules (2 rules)

Rule ID	Description	Justification	CVE Reference
`openpanel-security-bind-mounts`	Dangerous host path bind mounts	Container escape vector	CVE-2025-9074
`openpanel-security-secrets-in-env`	Secrets in ENV variables	Exposed via docker inspect	CVE-2025-3911

```
# openpanel-security-bind-mounts
Pattern: - /:/host|/proc:/|/sys:/|/etc:/|/var/run/docker.sock
Message: "Dangerous bind mount: mounting sensitive host paths enables container escape"
Fix: "Use named volumes or limit mounts to specific application directories"

# openpanel-security-secrets-in-env
Pattern: (?:PASSWORD|SECRET|TOKEN|API_KEY).*=.*[a-zA-Z0-9]{8,}
Message: "Potential secret in environment variable (visible via docker inspect)"
Fix: "Use Docker secrets or external secret management (HashiCorp Vault, AWS Secrets
Manager)"
```

P1.4 Cross-Platform Security (1 rule)

Rule ID	Description	Justification
`security-insecure-random`	Weak random number generation for security	Predictable tokens, session IDs

```
// Cross-language patterns
PHP:    \b(?:rand|mt_rand|srand)\s*\(.*(?:token|session|password|key|salt)
Perl:   \brand\s*\(.*(?:token|session|password|key)
Python: random\.randint.*(?:token|session|password|key)

Message: "Weak RNG for security purposes: use cryptographically secure random"
```

```
Fix:
  PHP: "Use random_bytes() or random_int() for security tokens"
  Perl: "Use Crypt::Random::Source or /dev/urandom"
  Python: "Use secrets module instead of random"
```

Priority 2: PHP 8.2/8.3 Compatibility (6 rules)

Based on PHP deprecation research and eusonlito.github.io/php-changes-cheatsheet:

Rule ID	Description	PHP Version	Severity
`php-compat-dynamic-properties`	Dynamic properties without #[AllowDynamicProperties]	8.2	warning
`php-compat-utf8-encode`	utf8_encode()/utf8_decode() removed	8.2	error
`php-compat-dollar-brace`	"${var}" string interpolation deprecated	8.2	warning
`php-compat-null-to-internal`	Passing null to non-nullable internal function params	8.1	warning
`php-compat-implicit-float-conversion`	Implicit float to int conversions losing precision	8.1	info
`php-compat-serializable-interface`	Serializable interface deprecated	8.1	warning

Implementation Details:

```
// php-compat-dynamic-properties (PHP 8.2)
Pattern: \$this->\w+\s*=.*(?!__construct|__set)
Context: Class without #[AllowDynamicProperties] attribute
Message: "Dynamic property creation deprecated in PHP 8.2"
Fix: "Declare property in class or add #[\\AllowDynamicProperties] attribute"

// php-compat-utf8-encode (PHP 8.2)
Pattern: \b(?:utf8_encode|utf8_decode)\s*\(
Message: "utf8_encode()/utf8_decode() removed in PHP 8.2"
Fix: "Use mb_convert_encoding($str, 'UTF-8', 'ISO-8859-1')"

// php-compat-dollar-brace (PHP 8.2)
Pattern: "\$\{[^}]+\}"
Message: "${var} string interpolation deprecated in PHP 8.2"
Fix: "Use {$var} or concatenation instead"

// php-compat-null-to-internal (PHP 8.1)
Pattern: \b(?:str_contains|str_starts_with|str_ends_with|htmlspecialchars)\s*\(\s*null
Message: "Passing null to non-nullable parameter deprecated in PHP 8.1"
Fix: "Explicitly check for null: if ($str !== null) { htmlspecialchars($str); }"
```

```
// php-compat-serializable-interface (PHP 8.1)
Pattern: implements\s+.*\bSerializable\b
Message: "Serializable interface deprecated in PHP 8.1"
Fix: "Implement __serialize() and __unserialize() magic methods instead"
```

Priority 3: WHMCS 8.11-8.13 Specific (4 rules)

Based on WHMCS release notes and API changes:

Rule ID	Description	WHMCS Version	Severity
`whmcs-payment-gateway-deprecated`	Deprecated payment gateway modules	8.11+	warning
`whmcs-capsule-orm-migration`	Direct DB queries vs. Capsule ORM	8.0+	info
`whmcs-hook-facade`	Old-style add_hook vs. Hook facade	8.0+	info
`whmcs-strict-types`	Missing strict type declarations	8.0+	info

Implementation Details:

```
// whmcs-payment-gateway-deprecated (WHMCS 8.11+)
Pattern: paypalbsc|paypalcheckout|modulomoip
Message: "Deprecated payment gateway: PayPal Basic (8.11), Modulo Moip (8.13)"
Fix: "Migrate to PayPal Payments module or alternative gateway"

// whmcs-capsule-orm-migration (WHMCS 8.0+)
Pattern: \b(?:select_query|insert_query|update_query|full_query)\s*\(
Context: Could use Capsule ORM instead
Message: "Consider migrating to WHMCS Capsule ORM for modern database operations"
Fix: "use WHMCS\\Database\\Capsule; Capsule::table('table')->where(...)->get();"

// whmcs-hook-facade (WHMCS 8.0+)
Pattern: add_hook\s*\(\s*['"][^'"]+['"]
Message: "Consider using WHMCS Hook facade for cleaner hook registration"
Fix: "use WHMCS\\Module\\Hook; Hook::add('HookName', 1, function($vars) { });"

// whmcs-strict-types (WHMCS 8.0+)
Pattern: ^<\?php\s*(?!declare\(strict_types=1\))
Context: WHMCS module file
Message: "WHMCS 8.0+ modules benefit from declare(strict_types=1)"
Fix: "Add declare(strict_types=1); after opening <?php tag"
```

Priority 4: cPanel UAPI Migration (3 rules)

Based on cPanel API deprecation roadmap (API1 removed since v88):

Rule ID	Description	Severity
`perl-cpanel-api1-functions`	Specific API1 functions with UAPI equivalents	error
`perl-cpanel-api2-to-uapi`	API2 functions that have UAPI replacements	info
`perl-whm-api1-usage`	WHM API1 usage detection	warning

Implementation Details:

```
# perl-cpanel-api1-functions
Functions: Email::addpop, Email::listpops, Ftp::addftpuser, MySQL::adddb
Pattern: \$cpapi->(?:Email|Ftp|MySQL|Postgres|Park|SubDomain)::
Message: "cPanel API1 function removed in v88+. Use UAPI equivalent"
Fix mapping:
  Email::addpop      → Email::add_pop
  Email::listpops    → Email::list_pops_with_disk
  Ftp::addftpuser    → Ftp::add_ftp
  MySQL::adddb       → Mysql::create_database

# perl-cpanel-api2-to-uapi
Functions with UAPI replacements (advisory only)
Pattern: \$cpanel_api2->(?:Email|Ftp|DiskUsage)::
Message: "API2 function has UAPI equivalent for better performance"
Fix: "Consider migrating to UAPI for improved performance and features"

# perl-whm-api1-usage
Pattern: \bWHM::API\s*::\s*(?:accountsummary|createacct|listaccts)
Message: "WHM API1 usage detected. Migrating to WHM API1 or UAPI recommended"
Fix: "Use WHM API1 (JSON) or UAPI where available"
```

Priority 5: OpenPanel Container Security (6 rules)

Based on Docker CVE-2025-9074, CVE-2025-23266, and container security best practices:

Rule ID	Description	CVE Reference	Severity
`openpanel-security-host-network`	Host network mode bypasses isolation	-	error
`openpanel-security-pid-namespace`	Shared PID namespace allows process inspection	-	warning
`openpanel-security-readonly-rootfs`	Missing read-only root filesystem	-	info
`openpanel-security-image-layers`	Credentials in Docker image layers	CVE-2025-3911	error

Rule ID	Description	CVE Reference	Severity
`openpanel-security-no-healthcheck`	Missing container health monitoring	-	warning
`openpanel-security-latest-tag`	Using :latest tag (non-deterministic)	-	warning

Implementation Details:

```
# openpanel-security-host-network
Pattern: network_mode:\s*['"]?host['"]?
Message: "Host network mode bypasses Docker network isolation"
Fix: "Use bridge networking with explicit port mappings"

# openpanel-security-pid-namespace
Pattern: pid:\s*['"]?host['"]?
Message: "Shared PID namespace allows container to see host processes"
Fix: "Remove pid: host unless absolutely required for debugging"

# openpanel-security-readonly-rootfs
Pattern: services:.*(?!read_only:\s*true)
Message: "Container runs with writable root filesystem"
Fix: "Add read_only: true and use tmpfs for writable directories"

# openpanel-security-image-layers (Dockerfile)
Pattern: (?:RUN|ENV|ARG).*(?:PASSWORD|SECRET|TOKEN|API_KEY).*=
Message: "Potential secret in Docker layer (visible in image history)"
Fix: "Use multi-stage builds and --mount=type=secret for build secrets"

# openpanel-security-no-healthcheck (Dockerfile)
Pattern: ^(?!.*HEALTHCHECK)
Message: "No HEALTHCHECK defined (OpenPanel relies on health checks)"
Fix: "Add HEALTHCHECK --interval=30s CMD curl -f http://localhost/ || exit 1"

# openpanel-security-latest-tag
Pattern: FROM\s+\w+:latest|image:\s*\w+:latest
Message: "Using :latest tag creates non-deterministic builds"
Fix: "Pin specific version tag (e.g., nginx:1.25.3-alpine)"
```

Priority 6: Advanced Static Analysis (5 rules)

More sophisticated patterns for common hosting vulnerabilities:

Rule ID	Description	Severity	Complexity
`security-type-confusion`	Type confusion vulnerabilities (loose comparisons)	warning	Medium

Rule ID	Description	Severity	Complexity
`whmcs-csrf-token`	Missing CSRF token validation in forms	error	Medium
`security-race-condition`	Race conditions in file operations	warning	High
`security-insecure-session`	Insecure PHP session configuration	warning	Medium
`security-http-headers`	Missing security headers (CSP, X-Frame-Options)	info	Low

Implementation Details:

```
// security-type-confusion
Pattern: \b(?:==|!=).*\$(?:_GET|_POST|_REQUEST)\[
Context: Not using === strict comparison
Message: "Loose comparison with user input enables type juggling attacks"
Fix: "Use strict comparison (=== or !==) with user input"
Example: if ($_GET['admin'] == true) // vulnerable to ?admin=1 or ?admin[]=1

// whmcs-csrf-token
Pattern: <form.*method=["']post["'](?!.*csrf)
Context: WHMCS template or module
Message: "POST form missing CSRF token validation"
Fix: "Add {$smarty.session.token} hidden input and validate with check_token()"

// security-race-condition
Pattern: file_exists\s*\([^)]+\).*(?:fopen|file_put_contents)
Context: TOCTOU pattern (Time-of-check to time-of-use)
Message: "Potential race condition: check-then-use file pattern"
Fix: "Use fopen() with 'x' mode (exclusive creation) or atomic operations"

// security-insecure-session
Pattern: session_start\s*\(.*(?!secure.*httponly)
Message: "Insecure session configuration: missing secure/httponly flags"
Fix: "session_set_cookie_params(['secure' => true, 'httponly' => true, 'samesite' =>
'Strict'])"

// security-http-headers
Pattern: header\s*\(\s*["']Content-Security-Policy
Negation check in file (look for absence of CSP header in module output)
Message: "Missing Content-Security-Policy header"
Fix: "Add CSP header to prevent XSS: header('Content-Security-Policy: default-src
\\'self\\'');"
```

---

3. Rule Implementation Specifications

3.1 Rule Structure Template

Every rule implementation must follow this structure:

```
export const ruleName: Rule = {
  // Required fields
  id: 'category-platform-description',              // Unique identifier
  description: 'Human-readable description',        // < 100 chars
  severity: 'error' | 'warning' | 'info',          // Impact level
  category: 'security' | 'compatibility' | 'best-practice',
  platform: 'whmcs' | 'cpanel' | 'openpanel' | 'all',

  // Optional fields
  minPhpVersion: '8.0',                            // PHP version constraint
  minWhmcsVersion: '8.11',                         // WHMCS version constraint
  minCpanelVersion: '132',                         // cPanel version constraint

  // Detection logic
  check: (context: RuleContext): LintResult[] => {
    const { code, filePath } = context;
    const results: LintResult[] = [];
    const lines = code.split('\n');

    // Pattern matching logic
    for (let i = 0; i < lines.length; i++) {
      const match = lines[i].match(/pattern/);
      if (match && match.index !== undefined) {
        results.push({
          file: filePath,
          line: i + 1,
          column: match.index + 1,
          message: 'Issue description and impact',
          ruleId: 'category-platform-description',
          severity: 'error',
          category: 'security',
          fix: 'Concrete fix suggestion with code example',

          // Optional: Auto-fix action (M2 milestone)
          fixAction: {
            range: {
              startLine: i + 1,
              startCol: match.index + 1,
              endLine: i + 1,
              endCol: match.index + 1 + match[0].length,
            },
            replacement: 'fixed_code',
            description: 'Replace X with Y',
          },
        });
      }
    }

    return results;
```

```
  },
};
```

3.2 Detection Pattern Guidelines

Pattern Complexity Levels:

1. **Simple** (90% of rules): Single-line regex

```
const match = line.match(/\beach\s*\(/);
```

1. **Context-Aware** (8% of rules): Multi-line pattern with context

```
const contextBlock = lines.slice(i - 5, i + 5).join('\n');
if (/pattern/.test(line) && !/exception/.test(contextBlock)) {
  // Report issue
}
```

1. **Stateful** (2% of rules): Track state across lines

```
let inClassDefinition = false;
for (const line of lines) {
  if (/^class\s+/.test(line)) inClassDefinition = true;
  if (inClassDefinition && /pattern/.test(line)) {
    // Report issue
  }
}
```

3.3 False Positive Mitigation

Strategies to reduce false positives:

1. **Comment Detection**: Skip commented lines

```
if (/^\s*(?:\/\/|#|\/\*|\*)/.test(line)) continue;
```

1. **String Literal Detection**: Don't flag patterns in strings

```
// Exclude patterns within quoted strings
if (/'[^']*pattern[^']*'/.test(line)) continue;
```

1. **Placeholder Detection**: Skip example/placeholder values

```
if (value.includes('example') || value.includes('placeholder')) continue;
```

1. **File Type Validation**: Only check relevant files

```
    if (!filePath.endsWith('.php') && !filePath.endsWith('.module')) return [];
```

1. **Context Validation**: Require additional evidence

```
    const isModuleFile = /function\s+\w+_MetaData/.test(code);
    if (!isModuleFile) return [];
```

3.4 SARIF Integration

All rules must be compatible with SARIF 2.1.0 format for GitHub Code Scanning:

```
{
  "version": "2.1.0",
  "$schema": "https://json.schemastore.org/sarif-2.1.0.json",
  "runs": [{
    "tool": {
      "driver": {
        "name": "HostingLint",
        "rules": [{
          "id": "security-sql-injection",
          "shortDescription": { "text": "SQL injection vulnerability" },
          "fullDescription": { "text": "Potential SQL injection..." },
          "help": {
            "text": "Use parameterized queries...",
            "markdown": "## Fix\n\nUse parameterized queries..."
          },
          "properties": {
            "tags": ["security", "sql", "cwe-89"],
            "precision": "high",
            "security-severity": "8.0"
          }
        }]
      }
    },
    "results": [...]
  }]
}
```

---

4. Technical Implementation Strategy

4.1 Current Architecture

Regex-Based Static Analysis:

- **Advantages**: Fast, portable, no runtime required, safe (no code execution)
- **Limitations**: Cannot understand program semantics, limited context awareness

- **Coverage**: 90% of security and compatibility issues detectable with regex

Pattern Matching Approach:

```
// Single-line regex (most rules)
const match = line.match(/\beach\s*\(/);

// Multi-line context (complex rules)
const contextBlock = lines.slice(i - 10, i + 10).join('\n');
if (/pattern1/.test(line) && !/exception/.test(contextBlock)) {
  // Rule violation
}

// File-level analysis (structural rules)
const hasMetaData = /function\s+\w*_MetaData\s*\(/.test(code);
```

4.2 Enhancement Opportunities (M2 Milestone)

AST Parsing for Advanced Rules:

For PHP (using php-parser via WASM):

```
import * as phpParser from 'php-parser';

const parser = new phpParser.Engine({
  parser: { extractDoc: true },
  ast: { withPositions: true },
});

const ast = parser.parseCode(code, 'module.php');
// Traverse AST to detect complex patterns
```

Benefits:

- **Semantic understanding**: Distinguish between variable assignments and comparisons
- **Scope analysis**: Track variable definitions and usage across functions
- **Type inference**: Detect type confusion more accurately

Trade-offs:

- **Performance**: AST parsing is 10-50x slower than regex
- **Complexity**: Requires parser maintenance and language-specific handling
- **Portability**: Need separate parsers for PHP, Perl, Python

Recommendation for M1: Stick with regex-based approach (proven, fast, sufficient for 50+ rules)

Recommendation for M2: Evaluate AST parsing for 10-15 advanced rules

4.3 Performance Optimization Strategies

Target: < 100ms for 1000 lines of code

1. **Early Returns**: Skip irrelevant files

```
if (!filePath.endsWith('.php')) return [];
```

1. **Compiled Regex**: Pre-compile patterns outside loop

```
const pattern = /\beach\s*\(/;  // Compile once
for (const line of lines) {
  if (pattern.test(line)) { ... }  // Reuse compiled regex
}
```

1. **Short-Circuit Evaluation**: Most efficient checks first

```
// Fast check first
if (!code.includes('eval')) return [];
// Expensive regex only if fast check passes
const matches = code.match(/\beval\s*\(/g);
```

1. **Parallel Execution**: Run rules concurrently (future enhancement)

```
const results = await Promise.all(
  rules.map(rule => rule.check(context))
);
```

1. **Incremental Analysis**: Only re-analyze changed files (IDE integration)

Benchmarking Plan:

- Baseline: Current 28 rules on 1000-line WHMCS module
- Target: 50 rules should maintain < 200ms total analysis time
- Test suite: Run against `examples/vulnerable/` modules in CI

---

5. Grant Milestone Mapping

M1: Rule Engine Expansion (120 hours, €6,000)

Deliverables:

- Implement 22 new rules (8 Priority 1, 6 Priority 2, 4 Priority 3, 3 Priority 4, 1 Priority 5)
- Achieve **50+ total rules** across all platforms
- 100% test coverage for all new rules
- Vulnerable examples for all new rules in `examples/vulnerable/`
- Updated documentation with rule catalog

Task Breakdown:

- **40 hours**: Priority 1 rules (high-impact security)
- **30 hours**: Priority 2 rules (PHP 8.2/8.3 compatibility)
- **20 hours**: Priority 3 + 4 rules (WHMCS, cPanel UAPI)
- **15 hours**: Test suite expansion and vulnerable examples
- **15 hours**: Documentation and rule catalog generation

Success Criteria:

- All 22 new rules pass test suite with 0 false positives on known-good code
- Detection accuracy validated against `examples/vulnerable/`
- Performance benchmark: < 200ms for 1000 lines
- SARIF output compatible with GitHub Code Scanning

M2: Auto-Fix and Plugin System (140 hours, €7,000)

Rule Enhancement:

- Add `fixAction` to 30 rules (current + new)
- Implement safe automated fixes for common issues
- Interactive fix preview in CLI

Rules with High Auto-Fix Potential:

Rule	Fix Type	Complexity
`php-compat-each`	Replace with foreach	Low
`php-compat-mysql-functions`	Replace with mysqli_	Low
`php-compat-dollar-brace`	Replace with {$var}	Low
`php-compat-utf8-encode`	Replace with mb_convert_encoding	Medium
`whmcs-metadata`	Insert MetaData() function template	Medium
`perl-strict-warnings`	Insert use strict/warnings	Low
`security-hardcoded-credentials`	Replace with env var placeholder	Medium

M3: Developer Experience (100 hours, €5,000)

IDE Integration:

- VS Code extension with inline rule warnings
- PhpStorm plugin via Language Server Protocol
- Real-time analysis as-you-type

M4: Documentation (80 hours, €4,000)

Rule Documentation Website:

- Auto-generated rule catalog from code
- Each rule page with: description, examples, fix, SARIF output
- Searchable by platform, category, severity
- Migration guides (PHP 7 → 8, API1 → UAPI, etc.)

---

6. Testing Strategy

6.1 Test Requirements

Every rule must have:

1. **Positive Test** (detects issue when present)

```
it('should detect SQL injection with $_GET in query', () => {
  const code = `$query = "SELECT * FROM users WHERE id = " . $_GET['id'];`;
  const results = analyze(code, 'module.php');
  expect(results).toHaveLength(1);
  expect(results[0].ruleId).toBe('security-sql-injection');
});
```

1. **Negative Test** (no false positive on safe code)

```
it('should not flag parameterized query', () => {
  const code = `$stmt = $pdo->prepare('SELECT * FROM users WHERE id = ?');`;
  const results = analyze(code, 'module.php');
  expect(results).toHaveLength(0);
});
```

1. **Edge Case Tests** (boundary conditions)

```
it('should not flag SQL in comments', () => {
  const code = `// $query = "SELECT * FROM users WHERE id = " . $_GET['id'];`;
  const results = analyze(code, 'module.php');
  expect(results).toHaveLength(0);
});
```

6.2 Vulnerable Example Files

Each rule should have a corresponding vulnerable example in `examples/vulnerable/`:

```
examples/vulnerable/
├── whmcs-vulnerable/
│   ├── sql_injection.php          # security-sql-injection
│   ├── php82_deprecations.php     # php-compat-* rules
│   ├── weak_crypto.php            # security-php-weak-crypto
│   └── ...
```

```
├── cpanel-vulnerable/
│   ├── api1_usage.pl              # perl-cpanel-api1-functions
│   ├── command_injection.pl       # security-perl-cmd-injection
│   └── ...
└── openpanel-vulnerable/
    ├── privileged-container.yml   # openpanel-security-capabilities
    ├── secrets-in-env.yml         # openpanel-security-secrets-in-env
    └── ...
```

6.3 Coverage Requirements

Target: 100% line coverage for rule files

```
npm run test:ci
# Must show 100% coverage for:
# - packages/core/src/rules/**/*.ts
```

Current coverage (28 rules):

```
File                | % Stmts | % Branch | % Funcs | % Lines
--------------------|---------|----------|---------|--------
rules/php/          |   100   |   100    |   100   |   100
rules/perl/         |   100   |   100    |   100   |   100
rules/openpanel/    |   100   |   100    |   100   |   100
rules/common/       |   100   |   100    |   100   |   100
```

6.4 Integration Testing

Test against real-world modules:

1. **WHMCS Sample Modules** (from marketplace)
2. Test against 5-10 popular WHMCS modules
3. Verify no false positives on production code

1. **cPanel Plugins** (from GitHub)
2. Test against cPanel plugin repositories
3. Validate API1 detection accuracy

1. **OpenPanel Extensions** (from DigitalOcean marketplace)
2. Test against sample OpenPanel Docker compose files
3. Verify container security rule accuracy

---

7. Performance Considerations

7.1 Performance Benchmarks

Current Baseline (28 rules):

- 1000-line PHP file: ~50ms
- 1000-line Perl file: ~45ms
- Docker Compose YAML: ~10ms

Target with 50+ rules:

- Maximum 2x slowdown: < 100ms for 1000 lines
- Parallelization goal: < 150ms for 5000-line project

7.2 Profiling Strategy

```
# Profile rule execution time
npm run benchmark -- --rules=all --file=examples/large-module.php

# Output:
# security-sql-injection:     2.3ms
# php-compat-each:            0.8ms
# whmcs-metadata:             1.2ms
# ... (total: 87ms for 50 rules)
```

7.3 Optimization Opportunities

1. **Rule Ordering**: Run fast rules first (early detection)
2. **Lazy Evaluation**: Skip expensive rules if fast rules already found issues
3. **Caching**: Cache compiled regex patterns
4. **Worker Threads**: Parallel analysis for large projects (future)

---

8. Community Contribution Model

8.1 Rule Submission Process

Template: `.github/RULE_TEMPLATE.md`

```
## Rule Proposal

### Rule Information
- **Rule ID**: `category-platform-description`
- **Category**: security | compatibility | best-practice
- **Platform**: whmcs | cpanel | openpanel | all
- **Severity**: error | warning | info

### Description
[Clear description of what this rule detects]

### Rationale
```

```
[Why is this rule important? Link to CVE, documentation, or best practices]

### Detection Pattern
[Regex or pattern description]

### Example Vulnerable Code
```

// Code that should trigger the rule

```
### Example Fixed Code
```

// Code that should NOT trigger the rule

```
### False Positive Considerations
[What are potential false positives and how to avoid them?]

### Related Rules
[Links to similar existing rules]

### References
- [CVE-XXXX-XXXX](https://cve.mitre.org)
- [OWASP Top 10](https://owasp.org)
```

8.2 Validation Criteria

Rules will be accepted if:

1. ✅ Addresses real security vulnerability or compatibility issue
2. ✅ Has < 5% false positive rate on test corpus
3. ✅ Includes 2+ test cases (positive + negative)
4. ✅ Provides concrete fix suggestion
5. ✅ Does not duplicate existing rule
6. ✅ Performance impact < 5ms per 1000 lines

8.3 Plugin System (M2 Milestone)

Allow community-contributed rules via npm packages:

```
// Custom rule package: @hostinglint-rules/my-company
export const myCompanyRules: Rule[] = [
  {
    id: 'my-company-license-check',
    description: 'Enforce company license header',
    // ... rule implementation
  },
];
```

```
// Load in config:
// .hostinglintrc.js
module.exports = {
  plugins: ['@hostinglint-rules/my-company'],
};
```

---

9. Appendices

9.1 Complete Rule Matrix (28 Current + 22 Proposed = 50 Total)

#	Rule ID	Category	Platform	Severity	Status
1	php-compat-each	compatibility	whmcs	error	✅ Current
2	php-compat-create-function	compatibility	whmcs	error	✅ Current
3	php-compat-mysql-functions	compatibility	whmcs	error	✅ Current
4	php-compat-curly-braces	compatibility	whmcs	error	✅ Current
5	security-sql-injection	security	whmcs	error	✅ Current
6	security-xss	security	whmcs	error	✅ Current
7	security-path-traversal	security	whmcs	error	✅ Current
8	whmcs-metadata	compatibility	whmcs	warning	✅ Current
9	whmcs-deprecated	compatibility	whmcs	warning	✅ Current
10	whmcs-hook-error-handling	best-practice	whmcs	warning	✅ Current
11	whmcs-config-function	compatibility	whmcs	warning	✅ Current
12	whmcs-license-check	best-practice	whmcs	info	✅ Current
13	whmcs-return-format	best-practice	whmcs	warning	✅ Current
14	perl-cpanel-api-version	compatibility	cpanel	warning	✅ Current
15	perl-deprecated-modules	compatibility	cpanel	warning	✅ Current
16	perl-security-taint	security	cpanel	error	✅ Current
17	perl-file-permissions	security	cpanel	warning	✅ Current
18	perl-input-validation	security	cpanel	warning	✅ Current
19	perl-strict-warnings	best-practice	cpanel	warning	✅ Current
20	perl-error-handling	best-practice	cpanel	warning	✅ Current
21	openpanel-api-versioning	compatibility	openpanel	warning	✅ Current

#	Rule ID	Category	Platform	Severity	Status
22	openpanel-security-capabilities	security	openpanel	error	✅ Current
23	openpanel-cli-validation	security	openpanel	warning	✅ Current
24	openpanel-dockerfile	best-practice	openpanel	warning	✅ Current
25	openpanel-resource-limits	best-practice	openpanel	warning	✅ Current
26	security-hardcoded-credentials	security	all	error	✅ Current
27	security-eval-usage	security	all	warning	✅ Current
28	best-practice-todo-fixme	best-practice	all	info	✅ Current
29	**security-php-deserialization**	**security**	**whmcs**	**error**	🔵 **P1**
30	**security-php-ssrf**	**security**	**whmcs**	**error**	🔵 **P1**
31	**security-php-weak-crypto**	**security**	**whmcs**	**error**	🔵 **P1**
32	**security-perl-sql-injection**	**security**	**cpanel**	**error**	🔵 **P1**
33	**security-perl-cmd-injection**	**security**	**cpanel**	**error**	🔵 **P1**
34	**openpanel-security-bind-mounts**	**security**	**openpanel**	**error**	🔵 **P1**
35	**openpanel-security-secrets-in-env**	**security**	**openpanel**	**error**	🔵 **P1**
36	**security-insecure-random**	**security**	**all**	**error**	🔵 **P1**
37	**php-compat-dynamic-properties**	**compatibility**	**whmcs**	**warning**	🟢 **P2**
38	**php-compat-utf8-encode**	**compatibility**	**whmcs**	**error**	🟢 **P2**
39	**php-compat-dollar-brace**	**compatibility**	**whmcs**	**warning**	🟢 **P2**
40	**php-compat-null-to-internal**	**compatibility**	**whmcs**	**warning**	🟢 **P2**
41	**php-compat-implicit-float-conversion**	**compatibility**	**whmcs**	**info**	🟢 **P2**
42	**php-compat-serializable-interface**	**compatibility**	**whmcs**	**warning**	🟢 **P2**
43	**whmcs-payment-gateway-deprecated**	**compatibility**	**whmcs**	**warning**	🟡 **P3**
44	**whmcs-capsule-orm-migration**	**best-practice**	**whmcs**	**info**	🟡 **P3**
45	**whmcs-hook-facade**	**best-practice**	**whmcs**	**info**	🟡 **P3**
46	**whmcs-strict-types**	**best-practice**	**whmcs**	**info**	🟡 **P3**
47	**perl-cpanel-api1-functions**	**compatibility**	**cpanel**	**error**	🟠 **P4**
48	**perl-cpanel-api2-to-uapi**	**compatibility**	**cpanel**	**info**	🟠 **P4**
49	**perl-whm-api1-usage**	**compatibility**	**cpanel**	**warning**	🟠 **P4**
50	**openpanel-security-host-network**	**security**	**openpanel**	**error**	🔴 **P5**

(Priority 5 & 6 rules truncated for brevity - full list available in GitHub issue tracker)

9.2 CVE Reference Mapping

CVE	Description	Related Rule(s)
CVE-2024-4577	PHP CGI RCE vulnerability	`php-compat-*` (upgrade context)
CVE-2025-9074	Docker Desktop container escape	`openpanel-security-bind-mounts`
CVE-2025-3911	Docker Desktop log file leak	`openpanel-security-secrets-in-env`
CVE-2025-23266	NVIDIA Container Toolkit escape	`openpanel-security-capabilities`
CVE-2025-31133	runC vulnerability	`openpanel-security-capabilities`
CWE-89	SQL Injection	`security-sql-injection`, `security-perl-sql-injection`
CWE-79	Cross-Site Scripting	`security-xss`
CWE-78	OS Command Injection	`perl-security-taint`, `security-perl-cmd-injection`
CWE-22	Path Traversal	`security-path-traversal`
CWE-798	Hardcoded Credentials	`security-hardcoded-credentials`
CWE-327	Weak Crypto	`security-php-weak-crypto`
CWE-502	Deserialization	`security-php-deserialization`
CWE-918	SSRF	`security-php-ssrf`

9.3 Version Compatibility Matrix

Platform	Supported Versions	Min Version	Max Version	Notes
PHP	7.4, 8.0, 8.1, 8.2, 8.3	7.4	8.3	PHP 7.4 EOL: Nov 2022
WHMCS	8.11, 8.12, 8.13	8.11	8.13+	8.11 EOL: Sep 2025
cPanel	v132, v133, v134	132	134+	Rolling release model
OpenPanel	1.0+	1.0	latest	Docker-based, version in manifest
Perl	5.36+	5.36	5.40+	cPanel bundles Perl

9.4 Tool Comparison Matrix

Feature	HostingLint	PHPStan	Perl::Critic	Hadolint
Hosting-Specific Rules	✅ Yes	❌ No	❌ No	❌ No
WHMCS API Detection	✅ Yes	❌ No	-	-
cPanel API Detection	✅ Yes	-	❌ No	-
Docker Security	✅ Yes	-	-	⚠️ Generic
Multi-Language	✅ PHP+Perl+Docker	❌ PHP only	❌ Perl only	❌ Docker only
No Runtime Required	✅ Yes	❌ Needs PHP	❌ Needs Perl	✅ Yes
SARIF Output	✅ Yes	✅ Yes	❌ No	✅ Yes
Auto-Fix	➡️ M2	❌ No	❌ No	❌ No
CI/CD Integration	✅ GitHub Actions	✅ Yes	✅ Yes	✅ Yes

Key Differentiator: HostingLint is the **only tool** that understands hosting control panel APIs and security patterns across PHP, Perl, and containerized environments.

---

10. Conclusion and Next Steps

10.1 Summary

This roadmap outlines a path to expand HostingLint from **28 to 50+ rules** across three hosting platforms:

- **8 Priority 1 rules**: High-impact security vulnerabilities (2024-2025 CVEs)
- **6 Priority 2 rules**: PHP 8.2/8.3 compatibility issues
- **4 Priority 3 rules**: WHMCS 8.11-8.13 specific patterns
- **3 Priority 4 rules**: cPanel UAPI migration detection
- **6 Priority 5 rules**: OpenPanel container security
- **5 Priority 6 rules**: Advanced static analysis patterns

10.2 Implementation Timeline (M1 Milestone)

Week 1-2 (40 hours): Priority 1 Security Rules

- Implement 8 high-impact security rules
- Create vulnerable examples for testing
- Validate against known CVEs

Week 3-4 (30 hours): PHP 8.2/8.3 Compatibility

- Implement 6 PHP deprecation rules
- Test against real WHMCS modules

- Document migration patterns

Week 5-6 (30 hours): Platform-Specific Rules

- WHMCS 8.11-8.13 rules (4)
- cPanel UAPI migration rules (3)
- OpenPanel container rules (3)

Week 7-8 (20 hours): Testing & Documentation

- Achieve 100% test coverage
- Update rule catalog
- Performance benchmarking

10.3 Success Metrics

Technical Metrics:

- ✅ 50+ total rules implemented
- ✅ 100% test coverage on rule files
- ✅ < 200ms analysis time for 1000 lines
- ✅ < 5% false positive rate
- ✅ SARIF compatibility for GitHub Code Scanning

Grant Application Metrics:

- ✅ Comprehensive technical documentation
- ✅ Evidence of community value (vulnerable examples)
- ✅ Clear roadmap through v1.0 release
- ✅ Differentiation from existing tools

10.4 Long-Term Vision

Beyond M1 (50+ rules):

- **M2 (Auto-Fix)**: Add automated fixes to 30 rules
- **M3 (IDE Integration)**: Real-time analysis in VS Code, PhpStorm
- **M4 (Documentation)**: Auto-generated rule catalog website
- **v1.0 Release**: Stable API, plugin system, community rule marketplace

Community Growth:

- Open rule submission process
- Monthly releases with community-contributed rules
- Integration with hosting industry tools (cPanel DevKit, WHMCS Marketplace)
- Presentation at hosting industry conferences

Document Prepared By: HostingLint Development Team

For: NLnet NGI Zero Commons Fund Application

Last Updated: February 8, 2026

Next Review: March 1, 2026 (pre-submission)

This roadmap is a living document and will be updated as the project evolves. Community feedback is welcome via GitHub Issues.